Task 4 – Order Management Database Documentation

Database: OrderManagementDB | **Prepared by:** Shree Pranava Ganesh | **Date:** 28 October 2026

Objective

This task extends the Product & Category Management database into a complete **order management system**. It adds the Customer, Orders and Order_Details tables, links them to Product and Category, and then uses SQL (INSERT, UPDATE, DELETE, JOINs, GROUP BY, aggregate functions) to produce a customer order history and a purchase analysis report.

---

1. Orders & Order_Details Table Design

1.1 Why two tables?

An order has a **header** (who ordered, when, the status, the total) and one or more **line items** (which products, how many, at what price). Storing both in one table would repeat the customer and date on every line. The standard design splits them:

- Orders stores one row per order (the header).
- Order_Details stores one row per product in an order (the line items).

This makes the relationship **one Order to many Order_Details**, and it resolves the **many-to-many** relationship between Orders and Products: one order can contain many products, and one product can appear in many orders. Order_Details is the **junction (bridge) table**.

1.2 Table: Customer

Column	Data Type	Constraints	Purpose
Customer_ID	INT	PRIMARY KEY	Unique customer identifier
Customer_Name	VARCHAR(100)	NOT NULL	Full name
Email	VARCHAR(100)	NOT NULL, UNIQUE	Contact email, no duplicates
Phone	VARCHAR(15)	UNIQUE	Contact number, no duplicates
City	VARCHAR(50)	NOT NULL	Customer location

1.3 Table: Orders

Column	Data Type	Constraints	Purpose
Order_ID	INT	PRIMARY KEY	Unique order identifier

Column	Data Type	Constraints	Purpose
Customer_ID	INT	NOT NULL, FOREIGN KEY → Customer(Customer_ID)	Who placed the order
Order_Date	DATE	NOT NULL	Date of the order
Total_Amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK (>= 0)	Sum of the order's line subtotals
Order_Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK IN ('Pending','Shipped','Delivered','Cancelled')	Order lifecycle stage

1.4 Table: Order_Details

Column	Data Type	Constraints	Purpose
Order_Detail_ID	INT	PRIMARY KEY	Unique line-item identifier
Order_ID	INT	NOT NULL, FOREIGN KEY → Orders(Order_ID)	Parent order
Product_ID	INT	NOT NULL, FOREIGN KEY → Product(Product_ID)	Product purchased
Quantity	INT	NOT NULL, CHECK (> 0)	Units purchased
Price	DECIMAL(10,2)	NOT NULL, CHECK (> 0)	Unit price at the time of purchase
(Order_ID, Product_ID)	–	UNIQUE (uq_order_product)	A product appears only once per order

1.5 Design decisions explained

- **Price is stored again in Order_Details.** Product prices change over time. Keeping a copy of the price on the line item preserves what the customer actually paid, so old orders never change when the catalogue price changes.
- **Subtotal is calculated, not stored.** `Quantity * Price` is computed in queries, which avoids inconsistent data.
- **Total_Amount is stored in Orders** as a convenience for fast reporting. It is refreshed by an `UPDATE ... SELECT SUM(Quantity * Price)` whenever line items change (see Section 4).
- **CHECK constraints** (positive price and quantity, non-negative total, valid status values) block invalid data at the database level. CHECK constraints are enforced from MySQL 8.0.16 onward.
- **Foreign keys** guarantee referential integrity: you cannot create an order for a customer who does not exist, or a line item for a missing product or order. For the same reason, line items must be deleted *before* their parent order.

1.6 Creation script (summary)

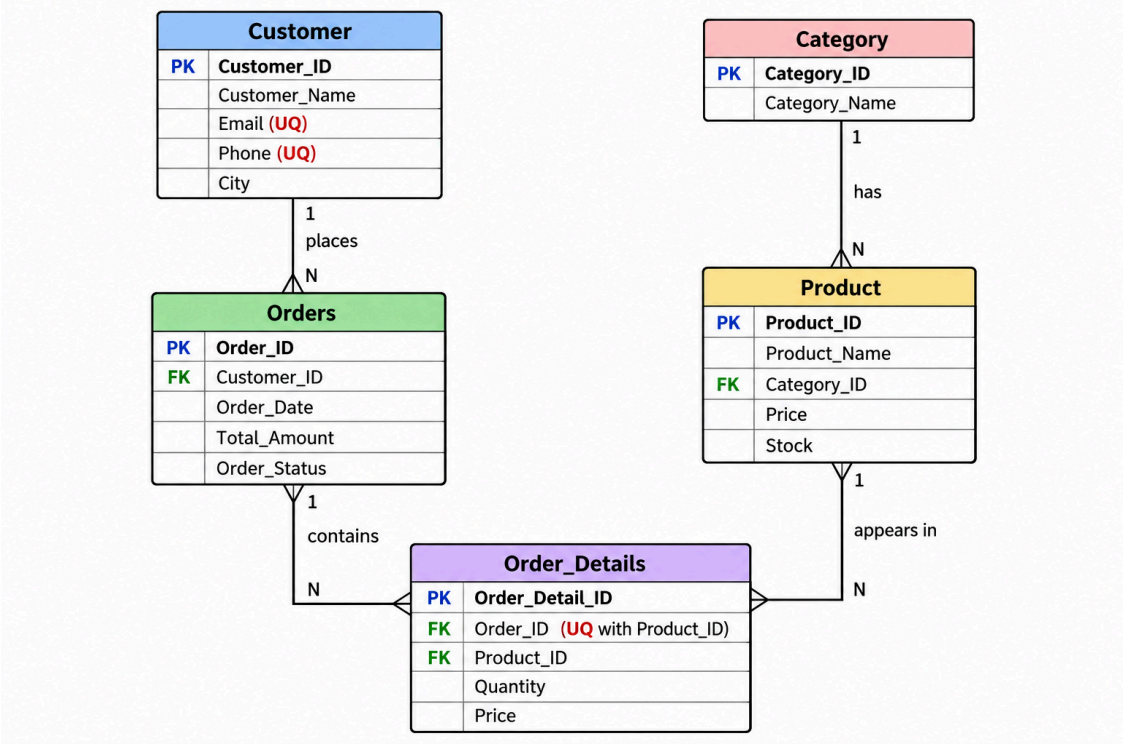
```
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY,
    Customer_ID INT NOT NULL,
    Order_Date DATE NOT NULL,
    Total_Amount DECIMAL(10,2) NOT NULL DEFAULT 0,
    Order_Status VARCHAR(20) NOT NULL DEFAULT 'Pending',
    CONSTRAINT fk_order_customer FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
    CONSTRAINT chk_total_amount CHECK (Total_Amount >= 0),
    CONSTRAINT chk_order_status CHECK (Order_Status IN ('Pending','Shipped','Delivered','Cancelled'))
);

CREATE TABLE Order_Details (
    Order_Detail_ID INT PRIMARY KEY,
    Order_ID INT NOT NULL,
    Product_ID INT NOT NULL,
    Quantity INT NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_detail_order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
    CONSTRAINT fk_detail_product FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID),
    CONSTRAINT chk_quantity CHECK (Quantity > 0),
    CONSTRAINT chk_order_price CHECK (Price > 0),
    CONSTRAINT uq_order_product UNIQUE (Order_ID, Product_ID)
);
```

---

2. Updated ER Diagram

The diagram shows all five tables. PK = primary key, FK = foreign key.



Relationships

Relationship	Cardinality	Foreign key	Meaning
Customer → Orders	1 : N	Orders.Customer_ID	One customer can place many orders
Orders → Order_Details	1 : N	Order_Details.Order_ID	One order has many line items
Product → Order_Details	1 : N	Order_Details.Product_ID	One product can appear in many line items
Category → Product	1 : N	Product.Category_ID	One category groups many products
Orders ↔ Product	M : N	via Order_Details	Resolved by the junction table

---

3. Sample Customer, Order & Order_Details Data

3.1 Customer

Customer_ID	Customer_Name	Email	Phone	City
1	Arun Kumar	arun@gmail.com	9876543210	Chennai
2	Priya Sharma	priya@gmail.com	9876543211	Coimbatore
3	Rahul Raj	rahul@gmail.com	9876543212	Madurai
4	Divya S	divya@gmail.com	9876543213	Trichy
5	Karthik R	karthik@gmail.com	9876543214	Thoothukudi
NULL	NULL	NULL	NULL	NULL

3.2 Category and Product (reference data)

Category_ID	Category_Name
2	Accessories
1	Electronics
3	Home Appliances
4	Stationery
NULL	NULL

Product_ID	Product_Name	Category_ID	Price	Stock
101	Laptop	1	55000.00	10
102	Smartphone	1	25000.00	20
103	Headphones	2	2000.00	30
104	Keyboard	2	1500.00	25
105	Mouse	2	800.00	40
106	Smart TV	3	35000.00	8
107	Mixer Grinder	3	4500.00	15
108	Notebook	4	100.00	100
109	Pen Set	4	250.00	80
110	Power Bank	2	1800.00	35
NULL	NULL	NULL	NULL	NULL

3.3 Orders

Order_ID	Customer_ID	Order_Date	Total_Amount	Order_Status
1001	1	2026-10-01	59000.00	Delivered
1002	2	2026-10-03	27300.00	Shipped
1003	1	2026-10-05	3800.00	Delivered
1004	3	2026-10-08	39500.00	Pending
1005	4	2026-10-10	2300.00	Delivered
1006	5	2026-10-12	5500.00	Shipped
1007	2	2026-10-15	3000.00	Pending
1008	1	2026-10-18	2050.00	Delivered
1009	3	2026-10-20	8200.00	Shipped
1010	4	2026-10-22	1500.00	Delivered
1011	5	2026-10-23	800.00	Cancelled
NULL	NULL	NULL	NULL	NULL

3.4 Order_Details

Order_Detail_ID	Order_ID	Product_ID	Quantity	Price
1	1001	101	1	55000.00
2	1001	103	2	2000.00
3	1002	102	1	25000.00
4	1002	104	1	1500.00
5	1002	105	1	800.00
6	1003	103	1	2000.00
7	1003	110	1	1800.00
8	1004	106	1	35000.00
9	1004	107	1	4500.00
10	1005	104	1	1500.00
11	1005	105	1	800.00
12	1006	107	1	4500.00
13	1006	108	10	100.00
14	1007	109	4	250.00
15	1007	103	1	2000.00
16	1008	105	1	800.00
17	1008	109	5	250.00
18	1009	103	2	2000.00
19	1009	105	3	800.00
20	1009	110	1	1800.00
21	1010	108	10	100.00
22	1010	109	2	250.00
NULL	NULL	NULL	NULL	NULL

3.5 Data operations performed

Operation	What the script does	Why
INSERT	Adds orders 1009, 1010 with Total_Amount = 0, then inserts their line items	The header is created first because Order_Details has a foreign key to it
UPDATE (total)	Sets Total_Amount = SUM(Quantity * Price) from Order_Details	Keeps the stored total consistent with the line items
UPDATE (status)	Moves order 1009 to Shipped; order 1010 from Pending → Shipped → Delivered	Demonstrates the order lifecycle
UPDATE (quantity)	Changes Notebook quantity in order 1010 from 5 to 10, then recalculates the total (1000 → 1500)	Shows that a line-item change must be followed by a total refresh
DELETE	Removes Cancelled order 1011: line items first, then the order	Child rows must be deleted before the parent because of the foreign key

---

4. Customer Order History & Purchase Analysis Report

All figures below are calculated from the final data in Section 3. The queries exclude Cancelled orders (none remain after the DELETE).

4.1 Customer order history

```
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date,
      o.Total_Amount, o.Order_Status
FROM Customer c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
ORDER BY c.Customer_ID, o.Order_Date;
```

Customer_ID	Customer_Name	Order_ID	Order_Date	Total_Amount	Order_Status
1	Arun Kumar	1001	2026-10-01	59000.00	Delivered
1	Arun Kumar	1003	2026-10-05	3800.00	Delivered
1	Arun Kumar	1008	2026-10-18	2050.00	Delivered
2	Priya Sharma	1002	2026-10-03	27300.00	Shipped
2	Priya Sharma	1007	2026-10-15	3000.00	Pending
3	Rahul Raj	1004	2026-10-08	39500.00	Pending
3	Rahul Raj	1009	2026-10-20	8200.00	Shipped
4	Divya S	1005	2026-10-10	2300.00	Delivered
4	Divya S	1010	2026-10-22	1500.00	Delivered
5	Karthik R	1006	2026-10-12	5500.00	Shipped
5	Karthik R	1011	2026-10-23	800.00	Cancelled

4.2 Customer purchase summary

```
SELECT c.Customer_ID, c.Customer_Name,
       COUNT(o.Order_ID) AS Total_Orders,
       SUM(o.Total_Amount) AS Total_Spending,
       ROUND(AVG(o.Total_Amount), 2) AS Average_Order_Value,
       MAX(o.Total_Amount) AS Highest_Order_Value
FROM Customer c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
WHERE o.Order_Status <> 'Cancelled'
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Spending DESC;
```

Customer_ID	Customer_Name	Total_Orders	Total_Spending	Average_Order_Value	Highest_Order_Value
1	Arun Kumar	3	64850.00	21616.67	59000.00
3	Rahul Raj	2	47700.00	23850.00	39500.00
2	Priya Sharma	2	30300.00	15150.00	27300.00
5	Karthik R	1	5500.00	5500.00	5500.00
4	Divya S	2	3800.00	1900.00	2300.00

4.3 Customer with the most orders

Both the `LIMIT 1` query and the `HAVING = MAX(...)` query return **Arun Kumar (Customer_ID 1) with 3 orders**. The `HAVING` version is the safer one: if two customers were tied, `LIMIT 1` would show only one of them, while `HAVING` would show both.

4.4 Sales by order status

```
SELECT Order_Status, COUNT(*) AS Number_Of_Orders, SUM(Total_Amount) AS Total_Amount
FROM Orders GROUP BY Order_Status;
```

Order_Status	Number_Of_Orders	Total_Amount
Delivered	5	68650.00
Shipped	3	41000.00
Pending	2	42500.00
Cancelled	1	800.00

4.5 Overall sales figures

Metric	Query used	Result
Total sales amount	SUM(Total_Amount) WHERE Order_Status <> 'Cancelled'	₹1,52,150.00
Number of orders	COUNT(*)	10
Average order value	ROUND(AVG(Total_Amount), 2)	₹15,215.00

Check: the five status totals (68,650 + 41,000 + 42,500) and the five customer totals (64,850 + 47,700 + 30,300 + 5,500 + 3,800) both add up to 1,52,150, so the report is internally consistent.

4.6 Product sales analysis

```
SELECT p.Product_ID, p.Product_Name,
    COUNT(od.Order_Detail_ID) AS Times_Ordered,
    SUM(od.Quantity) AS Quantity_Sold,
    SUM(od.Quantity * od.Price) AS Sales_Amount
FROM Product p
JOIN Order_Details od ON p.Product_ID = od.Product_ID
JOIN Orders o ON od.Order_ID = o.Order_ID
WHERE o.Order_Status <> 'Cancelled'
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Sales_Amount DESC;
```

Product_ID	Product_Name	Times_Ordered	Quantity_Sold	Sales_Amount
101	Laptop	1	1	55000.00
106	Smart TV	1	1	35000.00
102	Smartphone	1	1	25000.00
103	Headphones	4	6	12000.00
107	Mixer Grinder	2	2	9000.00
105	Mouse	4	6	4800.00
110	Power Bank	2	2	3600.00
104	Keyboard	2	2	3000.00
109	Pen Set	3	11	2750.00
108	Notebook	2	20	2000.00

Reading the result:

- **Highest revenue:** Laptop (₹55,000), then Smart TV and Smartphone. High-priced items dominate revenue even though they sell once each.
- **Most frequently ordered:** Headphones and Mouse (4 orders each), and both also sell 6 units.
- **Highest volume:** Notebook sells 20 units but earns the least revenue (₹2,000) because of its low unit price.
- **Every product has at least one sale**, so the `LEFT JOIN` queries for "never ordered" products return no zero rows.

4.7 Key findings

1. Total revenue is ₹1,52,150 across 10 orders from 5 customers.
2. Arun Kumar contributes about 42.6% of revenue (64,850 / 1,52,150).
3. Delivered orders account for ₹68,650; ₹83,500 is still in Pending or Shipped status and not yet fully realised.
4. Revenue is concentrated in Electronics (Laptop, Smart TV, Smartphone together = ₹1,15,000, about 75.6%).

Conclusion

The Orders and Order_Details tables, together with the foreign keys and CHECK constraints, give a normalised and validated order system. The SQL operations show the full life of an order: creation, line items, total calculation, status changes, correction and cancellation. The JOIN and GROUP BY queries turn this data into customer history and product-level sales insights.